

Environments and the working directory

Week 06 workshop

BAN405 Python Programming for Data Science

Why

Last week: your work cannot be lost

- Your BAN405 folder is a repository, and it is on GitHub.
- Every commit is a snapshot you can go back to.

Today's question

A classmate clones your repository tonight and runs your notebooks.

Do they run?

Not necessarily. git keeps your code, but your code is not the whole program.

Two lines, and what they depend on

Almost every notebook in the rest of the course starts like this:

```
import pandas as pd  
titanic = pd.read_csv("../data/titanic.csv")
```

Which Python is running?

Which packages does it have,
and which versions? That is the
environment.

Which folder is it running in?

Where does `../data/` start
from? That is the **working di-
rectory**.

Neither is written in your code. Today you learn to see both, and to control both.

Six exercises. Each builds on the one before.

	What you do	Where
1	Create an environment and use it	terminal, Positron
2	Write it to a file, delete it, rebuild it	terminal
3	Build the course environment	terminal, Positron
4	Read a data file from a notebook	Positron
5	Run someone else's project	Positron, terminal
6	Make your own repository runnable	Positron, GitHub

Explanations, options and fixes are in the [conda guide](#), in the `guides` folder.

Environments

“Python” is not one thing on your laptop

Ask the room what `import pandas` gives:

- `ModuleNotFoundError`
 - an older pandas, from an earlier course
 - a newer pandas, installed for something else
 - a Python that is not even Miniforge's
- Packages change between versions. Code that works with one can fail, or quietly give another answer, with the next.
 - Upgrade pandas for this course, and a project from an earlier one may stop working.

The course material is written for *one* set of versions. How do we all get it, without breaking whatever you already have?

An environment

- A **folder** with its own copy of Python and its own packages.
- One per project. Installing into one cannot change another.
- You switch with `conda activate`.
The prompt shows where you are:
`(base), (practice)`.

```
miniforge3/
├── python          ← base
├── envs/
│   ├── practice/
│   ├── ban405/
│   └── temperature-app/
```

base

The environment conda uses for itself. Whatever is in it already, leave it, and do not add to it.

Where packages come from: conda-forge

- **conda** downloads packages, and works out which versions fit together.
- It downloads them from a **channel**. Ours is **conda-forge**: a large collection, maintained by volunteers.
- **conda-forge.org/packages**: every package, and every version of it.

Together: before you start

Mac: **Terminal**. Windows: **Miniforge Prompt**. Never Positron's terminal.

```
conda --version  conda env list  conda config --show channels
```

conda-forge should be at the top. If it is not, the guide shows how to add it.

Exercise 1 — Create an environment and use it

EXERCISE

- **In the terminal:** create `practice`, with Python 3.14 and pandas. Compare it with `base`.
- **In Positron:** a notebook that prints which Python runs it, and which pandas.
- Run the same cell with `practice`, then with `base`.

Steps

All steps and checks are in [exercises.md](#). Mac: **Terminal**. Windows: **Miniforge Prompt**.

What you just saw

- The **terminal** is where environments are made.
- **Positron** is where you choose which one runs your code: the **kernel selector**.
- `sys.executable` tells you which Python is running. Its path ends in `envs/practice`.

The lesson

Same code, same computer, different result. It depends on which environment runs it.

`ModuleNotFoundError` for a package you know you installed? Wrong environment.

An environment as a file

```
name: practice
channels:
  - conda-forge
dependencies:
  - pandas
  - python=3.14
prefix: ...
```

```
conda env export --from-history
> practice.yml
```

- The packages *you asked for*, and where they come from.
- Anyone can rebuild it:

```
conda env create --file
practice.yml
```

The file is the recipe. The environment itself is disposable.

Exercise 2 — Write it down, delete it, rebuild it

EXERCISE

- Go to your `week06` folder with `cd`: copy the path, do not type it.
- Export `practice` to `practice.yml`. Delete the environment.
- Open a *new* terminal, and rebuild it from the file.

Steps

All steps and checks are in [exercises.md](#). One step fails on purpose. Read the error.

What went wrong, and why

- A terminal is always *in* a folder: its **working directory**. A new one starts in your home folder.
- `--file practice.yml` means “*practice.yml, in the folder I am in*”.
- The error names the exact path conda tried. The file was fine. The command ran in the wrong place.

Where am I?

Windows: the prompt shows it: `(base) C:\Users\you\BAN405\week06>`

Mac: the prompt shows the folder's name; `pwd` prints the whole path.

Keep this error in mind. You meet it again in a notebook, where no prompt tells you where you are.

Pinned versions

`python=3.14 pandas`

- “The newest that fits, today.”
- Build it next month, get something else.

`pandas=3.0.3`

- “Exactly this one, always.”
- Every laptop, every month, the same.

The course environment

`name: ban405`

`channels:`

- `conda-forge`
- `nodefaults`

`dependencies:`

- `python=3.14.6`
- `numpy=2.5.1`
- `pandas=3.0.3`
- `matplotlib=3.11.0`
- `statsmodels=0.14.6`
- ...

A number on the screen at the front of the room is the same number on your laptop.

Exercise 3 — Build the course environment

EXERCISE

- Build **ban405** straight from its web address: no download first.
- Switch **checks.ipynb** to the **ban405** kernel, and run it again.
- Compare its pandas with the one **practice** got.

Steps

All steps and checks are in [exercises.md](#). It takes a few minutes: read the file while it runs.

The working directory

Every running program is somewhere

A **relative path** like `data/titanic.csv` is read starting from the **working directory**.

Running in

a terminal

a notebook

a script, with ► in Positron

Working directory

where you `cd`'d to: the prompt shows it

the folder the notebook is saved in

the folder open in Positron

- `os.getcwd()` prints it from inside Python.
- `..` means *the folder above*.

Exercise 4 — Where your notebook looks for files

EXERCISE

- **Part A:** in `week06/checks.ipynb`, find the working directory, and read `titanic.csv` until it works.
- **Part B:** the same line, in a notebook at the top of your BAN405 folder.
- pandas and matplotlib today are a demonstration. They are the rest of the course.

Steps

All steps and checks are in [exercises.md](#).

Why the course writes ../data/

```
BAN405/  
├── data/  
│   └── titanic.csv  
└── week06/  
    └── checks.ipynb ← here
```

"../data/titanic.csv"

up one folder, into data

```
project/  
├── data/  
│   └── titanic.csv  
└── analysis.ipynb ← here
```

"data/titanic.csv"

straight into data

The rule

A path in a notebook is written **from the folder the notebook is saved in**. Know where the notebook is and where the data is, and you can always write it.

Four ways to point at a file

	Example	Works
Relative	<code>"../data/titanic.csv"</code>	wherever the project goes
Relative	<code>"data/titanic.csv"</code>	the same, for a notebook at the top
Absolute	<code>r"C:\Users\you\..."</code>	only on your own computer
Web address	<code>"https://..."</code>	anywhere, but only online

The course uses relative paths. A project keeps working when it moves, as long as the notebook and the data keep their places relative to each other.

The other two are an optional exercise at home.

Demo: a script is not a notebook

LIVE DEMO

1. In `week06`, a script `read_titanic.py` with the line that worked in the notebook:

```
pd.read_csv("../data/titanic.csv")
```

2. Run it with ▶. `FileNotFoundError`.
3. `print(os.getcwd())`: the `BAN405` folder, not `week06`.

Why

A script run with ▶ runs from the folder open in Positron, not from its own folder. The rest of the course is notebooks, so you will rarely meet this. But when you do, you know why.

Projects

Exercise 5 — Run someone else's project

EXERCISE

- Clone `temperature-app` into `git-practice`: your converter, as a web app.
- Try it in `ban405` first. Then follow its README.
- Convert 212 °F in your browser.

Steps

All steps and checks are in [exercises.md](#). The app keeps running until you press `Ctrl+C` in the terminal.

What makes a project runnable

```
temperature-app/  
├── .gitignore  
├── README.md      ← how to run  
├── app.py  
├── environment.yml ← what it needs  
└── temperature_tools.py
```

1. **The code**, in a repository.
2. **The data**, read with relative paths.
3. **An environment file**, with the versions that matter pinned.
4. **A README** that says how to run it.

You ran a project you had never seen, with a package you had never used, without installing anything by hand.

Exercise 6 — Make your own repository runnable

EXERCISE

- Add the course's `environment.yml` to the top of your BAN405 folder.
- Add a *How to run* section to your README.
- Commit, and push. Look at the result on GitHub.

Steps

All steps and checks are in [exercises.md](#).

From now on

- **pip** is Python's own installer, and it works inside a conda environment too.
- **Prefer conda.** It checks that all your packages fit together before installing anything.
- **Use pip when conda cannot help:** a package that is not on conda-forge.
- **conda first, then pip.** Installing with conda afterwards can undo what pip did.

Careful

--from-history does not record what pip installed. Add those packages to the file by hand.

Environments

- One environment per project.
- Leave **base** alone.
- Pin the versions that matter.
- Keep `environment.yml` next to the code.
- Delete environments you no longer use.

Every session from now on

1. Open your **BAN405 folder** in Positron.
2. Choose the **ban405** kernel.
3. Read data with `../data/`.
4. Commit as you work. Push when you finish.

What we did not cover

Worth knowing these exist:

- **conda install**: adding a package to an environment that already exists. The guide shows how.
- **venv and pip**: Python's own environments. The same idea, without conda; the file is usually `requirements.txt`.
- **Paths from the script's own folder**: `pathlib` and `__file__` make a script independent of where it is run from.
- **Lock files**: every package, down to the last dependency, pinned for each operating system.

What you did today

Today you

- created, deleted and rebuilt an environment
- built the course environment from a file
- found out where a notebook looks for files
- ran someone else's project from its README
- made your own repository runnable

At home (in [exercises.md](#))

- Finish Exercise 6, if needed.
- Delete `practice` and `temperature-app`. Keep **ban405**.
- *Optional*: absolute paths and web addresses; test your repository from a fresh clone.

Next week

Data. Reading a file into a table with pandas, and getting it into a shape you can trust.